Project Interim Report

On

GROCERY BILLING RECEIPT GENERATOR

(Using Core Java & OOP Concepts)

RU-100-01-00012

Object Oriented Programming with Java

SESSION: 2025-26

Submitted By

SHAILESH KUMAR GUPTA

RU-25-11326

Bachelor of Technology in Computer Science & Engineering in association with Google

Under the Guidance of **Mr. Ashish shivhare Sir**

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

RUNGTA INTERNATIONAL SKILLS UNIVERSITY

BHILAI, CG

(April 2026)

Table of Contents

Acknowledgement

I would like to express my sincere and heartfelt gratitude to my respected project guide, **Mr. Ashish shivhare sir**, for his invaluable guidance, constant encouragement, and continuous support throughout the entire duration of this project. His deep knowledge, constructive suggestions, and patient mentoring helped me to understand complex concepts with clarity and confidence. His timely feedback and motivation played a crucial role in the successful completion of this project.

I am also highly thankful to the esteemed faculty members of the School of Computer Science and Engineering, Rungta International Skills University, for providing me with the opportunity to undertake this project. Their academic support and encouragement have greatly contributed to enhancing my understanding of Object-Oriented Programming and its practical applications.

I would like to extend my sincere thanks to my friends and classmates for their cooperation, valuable suggestions, and continuous support during the development of this project. Their discussions and shared ideas helped me overcome various challenges and improve the overall quality of my work.

Finally, I would like to thank everyone who directly or indirectly contributed to the successful completion of this project.

SHAILESH KUMAR GUPTA

RU-25-11326

Abstract

The Grocery Billing Receipt Generator is a Java-based software system designed to provide an efficient and automated solution for generating grocery bills. In today's retail environment, manual billing is slow, error-prone, and difficult to scale. This project addresses these challenges by offering a structured, automated, and user-friendly approach to grocery billing and receipt generation.

The application enables users to add multiple grocery items with their names, prices, and quantities. Each item entry includes essential attributes such as item name, price per unit, and quantity purchased, ensuring accurate bill calculation. The system automatically computes the item-wise total, overall subtotal, applicable tax, and grand total — eliminating the need for manual computation.

One of the key features of the application is its 5% tax calculation applied on the subtotal, along with a neatly formatted receipt that is easy to read and professional in appearance. The system also performs input validation to prevent negative prices or zero quantities, ensuring data integrity throughout the billing process.

The project is implemented using Object-Oriented Programming (OOP) principles such as classes, objects, encapsulation, and aggregation. These concepts ensure that the application is well-structured, modular, and easy to maintain. The use of ArrayList allows dynamic storage of item records, while StringBuilder is used for efficient receipt generation.

The Grocery Billing Receipt Generator serves as a practical example of applying programming concepts to solve real-world retail problems. It is especially beneficial for small shop owners, students learning Java OOP, and developers looking for a billing system template. The system can be further improved by integrating GUI, database support, discount features, and barcode scanning.

In conclusion, the Grocery Billing Receipt Generator not only simplifies the billing process but also demonstrates the effective use of Java and Object-Oriented Programming concepts in building a functional and practical application.

Introduction

In today's rapidly evolving digital world, automation has become an essential part of everyday commercial activities. Small grocery stores and supermarkets often face challenges in managing billing efficiently. Traditional methods such as handwritten bills or simple calculators are slow, prone to errors, and difficult to maintain. To address these challenges, the Grocery Billing Receipt Generator has been developed as a simple yet effective solution using Java programming.

The Grocery Billing Receipt Generator is designed to help shopkeepers and users systematically generate bills for grocery purchases. It provides a user-friendly console interface where the user can add multiple items, specify their prices and quantities, and instantly generate a formatted receipt showing item-wise totals, subtotal, tax, and grand total.

One of the key features of this application is automatic tax calculation. The system applies a 5% tax on the subtotal and computes the grand total without any manual effort. This reduces errors and speeds up the billing process significantly, making it suitable for real-world retail scenarios.

Another important feature is input validation. The system ensures that prices are non-negative and quantities are positive, preventing erroneous entries from corrupting the bill. If invalid data is entered, the system displays an appropriate error message and prompts the user to re-enter the details.

The application is developed using Object-Oriented Programming (OOP) concepts such as classes, objects, encapsulation, and aggregation. These concepts ensure that the system is well-structured, modular, and easy to maintain. The Item class encapsulates item attributes and behavior, while the Bill class aggregates multiple Item objects and manages receipt generation using StringBuilder and ArrayList.

The Grocery Billing Receipt Generator is implemented as a console-based program, making it lightweight, easy to operate, and suitable for beginners as well as users with minimal technical knowledge. Despite its simplicity, the application demonstrates the practical use of programming concepts in solving real-world problems. It also provides a strong foundation for future enhancements such as GUI development, database integration, and discount management.

Overall, this project not only fulfills the basic requirement of grocery billing automation but also highlights the importance of clean code, OOP design, and input validation. By combining ease of use with powerful features, the Grocery Billing Receipt Generator offers a reliable and practical solution for automated billing.

Objectives

- To develop a user-friendly grocery billing system that allows shopkeepers to easily add items and generate receipts.
- To provide a structured way of storing item data including item name, price per unit, and quantity, ensuring accurate bill computation.
- To implement automatic calculation of item-wise totals, subtotal, 5% tax, and grand total, reducing manual effort.
- To generate a clean, formatted receipt that is easy to read and suitable for real-world billing scenarios.
- To perform input validation ensuring that prices are non-negative and quantities are positive, maintaining data integrity.
- To apply Object-Oriented Programming (OOP) concepts such as classes, objects, encapsulation, and aggregation in designing a real-world billing application.
- To utilize data structures like ArrayList for dynamic item storage and StringBuilder for efficient receipt construction.
- To reduce dependency on manual billing methods such as handwritten receipts or calculators, improving accuracy and speed.
- To design a system that is simple, scalable, and easy to maintain, allowing future enhancements and modifications.
- To provide a practical learning experience by implementing programming concepts in a real-life commercial problem scenario.
- To create a foundation for future improvements such as GUI development, discount modules, database connectivity, and barcode scanning.
- To demonstrate how Java OOP principles can be effectively used to solve everyday retail and billing problems.

Scope

The Grocery Billing Receipt Generator is developed to provide an efficient, structured, and user-friendly solution for automating the billing process in grocery stores. The scope of this project primarily focuses on small shop owners, students, and individuals who require a simple system to generate grocery bills and receipts. In its current form, the application is implemented as a console-based program using Java, making it lightweight, easy to operate, and suitable for beginners as well as users with minimal technical knowledge.

The system enables users to enter multiple grocery items with their respective names, prices, and quantities. It automatically calculates item-wise totals, the overall subtotal, applicable 5% tax, and the grand total payable. The receipt generated is neatly formatted and presents all billing details in a structured tabular layout, making it easy to read and understand.

From an academic perspective, the project demonstrates the practical implementation of Object-Oriented Programming (OOP) concepts such as classes, objects, encapsulation, and aggregation. It also utilizes dynamic data structures like ArrayList for storing multiple item records and StringBuilder for efficient receipt text generation. This makes the application not only functional but also a valuable learning tool for understanding programming concepts and their real-world applications.

However, the current scope of the project is limited to temporary in-memory data processing, meaning that all billing data is lost once the program is terminated. This limitation provides an opportunity for future enhancements. The application can be extended by integrating a database system such as MySQL or SQLite to enable permanent data storage and bill history retrieval. Additionally, a graphical user interface (GUI) can be developed using Java Swing or JavaFX to improve user experience.

Further improvements may include implementing features such as discount management, multiple tax brackets, barcode scanning integration, customer details capture, bill printing support, and graphical sales reports. With these enhancements, the Grocery Billing Receipt Generator can evolve into a comprehensive point-of-sale (POS) system suitable for both small shops and medium-scale retail businesses.

In conclusion, the scope of this project is not limited to basic billing but extends to providing a foundation for developing advanced retail management solutions, making it both practically useful and academically significant.

System Requirements

The Grocery Billing Receipt Generator requires a basic computing environment to run efficiently. Since it is a console-based Java application, it does not require high-end hardware or advanced software, making it accessible to a wide range of users.

Hardware Requirements

Minimum Requirements:

- Processor: Intel Pentium or equivalent
- RAM: 2 GB
- Storage: 100 MB free disk space
- Input Devices: Keyboard

Recommended Requirements:

- Processor: Intel i3 or above
- RAM: 4 GB or higher
- Storage: 500 MB or more free space
- Display: Standard monitor

Software Requirements

Minimum Requirements:

- Operating System: Windows 7 / Linux / macOS
- Java Development Kit (JDK): Version 8 or above
- Any Text Editor (Notepad / Edit Plus)

Recommended Requirements:

- Operating System: Windows 10/11
- Java Development Kit (JDK): Latest version (JDK 17 or above)
- IDE: VS Code / Eclipse / IntelliJ IDEA

Methodology

The Grocery Billing Receipt Generator follows a systematic and structured methodology to process user inputs and generate formatted bills efficiently. The application is designed using a sequential input-driven approach, allowing users to enter item details one by one before the receipt is generated. The overall workflow is divided into multiple steps, ensuring smooth execution and easy understanding.

Step 1: Initialization

The program begins by initializing necessary components — a Scanner for reading user input, a Bill object to manage item collection and receipt generation, and an ArrayList inside Bill to dynamically store Item objects.

Step 2: Input Number of Items

The user is prompted to enter the total number of grocery items to be billed. This determines how many item entry iterations the program will perform.

Step 3: Enter Item Details

For each item, the user enters the item name, price per unit, and quantity. The system validates the inputs — if the price is negative or quantity is zero/negative, an IllegalArgumentException is thrown and the user is asked to re-enter that item's details.

Step 4: Create Item Objects and Add to Bill

After successful validation, a new Item object is created with the provided details and added to the Bill's ArrayList. This process repeats for each item.

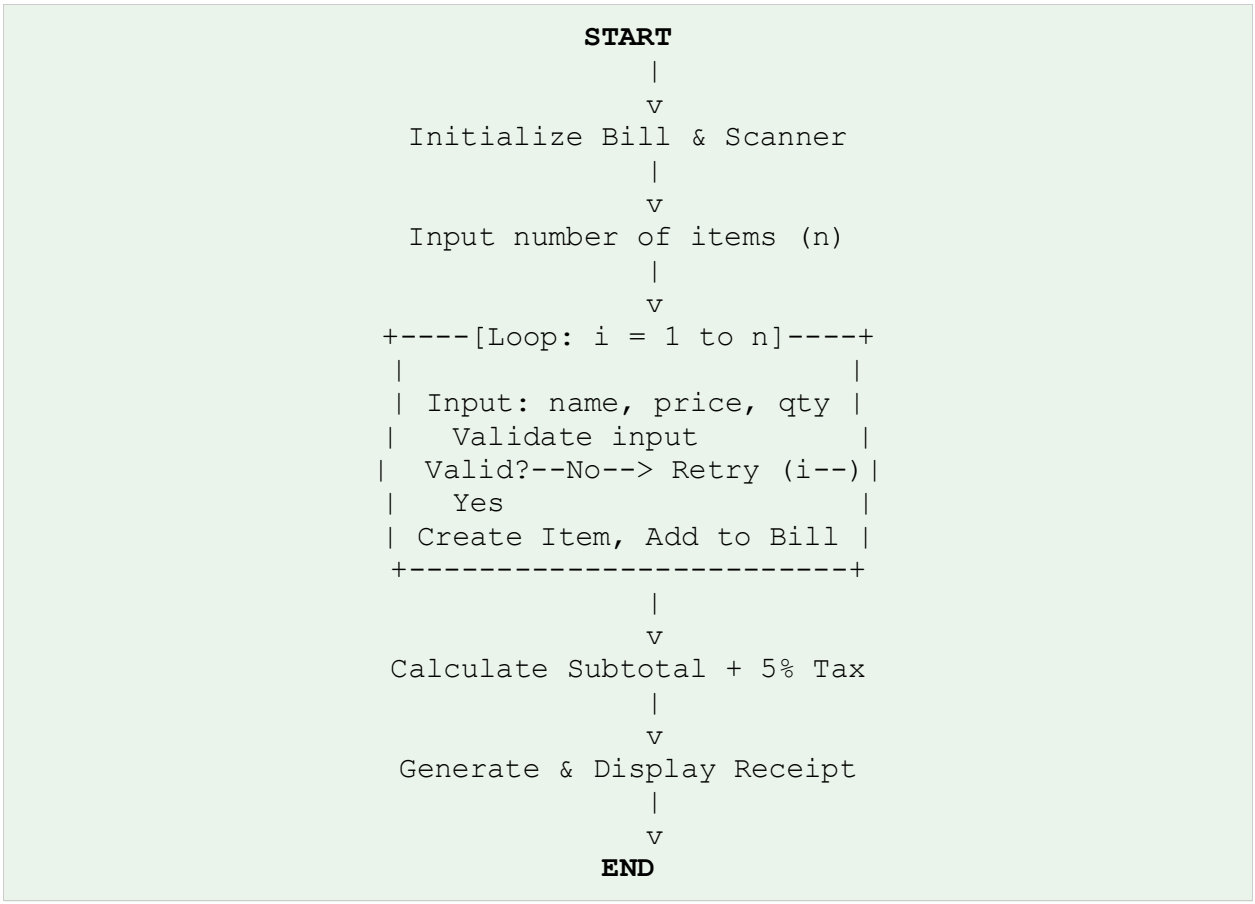
Step 5: Calculate Subtotal and Tax

Once all items are added, the Bill object calculates the subtotal by summing all item totals (price × quantity). A 5% tax is then computed on the subtotal.

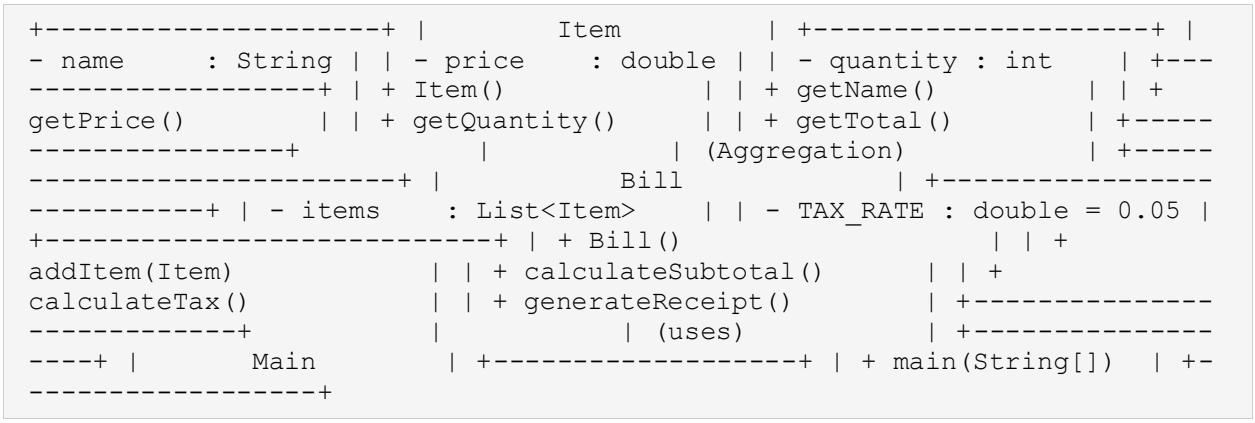
Step 6: Generate and Display Receipt

The generateReceipt() method of the Bill class builds a formatted receipt using StringBuilder. It displays item name, price, quantity, and total in a tabular format, followed by the subtotal, tax, and grand total. The receipt is then printed to the console.

Flowchart


START
Initialize Bill & Scanner
Input number of items (n)
[Loop: i = 1 to n]
Input: name, price, qty
Validate input
Valid?--No--> Retry (i--)
Yes
Create Item, Add to Bill
Calculate Subtotal + 5% Tax
Generate & Display Receipt
END

Class Diagram



Implementation

The Grocery Billing Receipt Generator is implemented using Java and follows Object-Oriented Programming (OOP) principles to ensure modularity, reusability, and easy maintenance. The system is divided into three core components: the Item class, the Bill class, and the Main class — each responsible for specific functionalities within the application.

1. Item Class

- The Item class represents a single grocery item in the billing system. It contains three private attributes: name (String), price (double), and quantity (int).
- A constructor is used to initialize these attributes with validation — if price is negative or quantity is zero or less, an IllegalArgumentException is thrown immediately.
- Getter methods (getName(), getPrice(), getQuantity()) provide controlled access to the private fields, demonstrating encapsulation.
- A getTotal() method computes and returns the total cost for that item (price × quantity), encapsulating item-level calculation logic.

2. Bill Class

- The Bill class acts as the main aggregator and controller. It contains a private List<Item> (ArrayList) that stores all Item objects added during billing.
- A static constant TAX_RATE (0.05) stores the 5% tax rate, ensuring a single source of truth for tax computation.
- The addItem(Item) method adds a validated Item object to the internal list.
- The calculateSubtotal() method iterates over all items and sums their individual totals to compute the overall subtotal.
- The calculateTax() method returns 5% of the subtotal.
- The generateReceipt() method uses StringBuilder to construct a professionally formatted receipt, displaying item details in tabular form followed by subtotal, tax, and grand total.

3. Main Class

- The Main class contains the main() method which serves as the program entry point.
- It creates a Scanner for reading user input and a Bill object to manage the billing session.
- The user is prompted to enter the number of items, followed by details for each item in a loop.
- A try-catch block handles IllegalArgumentException — if invalid input is detected, the user is shown an error message and the loop counter is decremented to retry that item.

- After all items are entered, generateReceipt() is called and the receipt is printed to the console.

4. Key OOP Concepts Applied

- Encapsulation: All attributes in Item are private with public getter methods.
- Aggregation: Bill aggregates multiple Item objects using ArrayList.
- Constructor: Item constructor enforces business rules (validation) at object creation.
- Abstraction: The receipt generation logic is hidden inside generateReceipt() — callers only see the output.

5. Source Code

```
import java.util.*;

class Item {
    private String name;
    private double price;
    private int quantity;

    public Item(String name, double price, int quantity) {
        if (price < 0 || quantity <= 0) {
            throw new IllegalArgumentException("Invalid price or quantity!");
        }
        this.name = name;
        this.price = price;
        this.quantity = quantity;
    }

    public String getName() {
        return name;
    }

    public double getPrice() {
```

```
        return price;
    }

    public int getQuantity() {
        return quantity;
    }

    // Calculate total for this item
    public double getTotal() {
        return price * quantity;
    }
}

class Bill {
    private List<Item> items;
    private static final double TAX_RATE = 0.05; // 5% tax

    public Bill() {
        items = new ArrayList<>();
    }

    public void addItem(Item item) {
        items.add(item);
    }

    public double calculateSubtotal() {
        double total = 0;
        for (Item item : items) {
            total += item.getTotal();
        }
        return total;
    }
```

```
    public double calculateTax() {
        return calculateSubtotal() * TAX_RATE;
    }

    public String generateReceipt() {
        StringBuilder receipt = new StringBuilder();

        receipt.append("\n------ Grocery Bill ------\n");
        receipt.append(String.format("%-10s %-10s %-10s %-10s\n", "Item", "Price", "Qty",
"Total"));

        for (Item item : items) {
            receipt.append(String.format("%-10s %-10.2f %-10d %-10.2f\n",
                    item.getName(),
                    item.getPrice(),
                    item.getQuantity(),
                    item.getTotal()));
        }

        double subtotal = calculateSubtotal();
        double tax = calculateTax();
        double grandTotal = subtotal + tax;

        receipt.append("\nSubtotal: ").append(String.format("%.2f", subtotal));
        receipt.append("\nTax (5%): ").append(String.format("%.2f", tax));
        receipt.append("\nGrand Total: ").append(String.format("%.2f", grandTotal));
        receipt.append("\n--------------------------\n");

        return receipt.toString();
    }
}

public class Main {
```

```
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    Bill bill = new Bill();

    System.out.print("Enter number of items: ");
    int n = sc.nextInt();
    sc.nextLine();

    for (int i = 0; i < n; i++) {
        System.out.println("\nEnter details for item " + (i + 1));

        System.out.print("Item name: ");
        String name = sc.nextLine();

        System.out.print("Price: ");
        double price = sc.nextDouble();

        System.out.print("Quantity: ");
        int quantity = sc.nextInt();
        sc.nextLine();

        try {
            Item item = new Item(name, price, quantity);
            bill.addItem(item);
        } catch (IllegalArgumentException e) {
            System.out.println("Error: " + e.getMessage());
            i--;
        }
    }

    System.out.println(bill.generateReceipt());

    sc.close();
```

```
  }
}
```

6. Sample Output

```
Enter number of items: 3  Enter details for item 1 Item name: Rice Price:
50 Quantity: 2  Enter details for item 2 Item name: Milk Price: 30
Quantity: 3  Enter details for item 3 Item name: Sugar Price: 45 Quantity:
1  ------ Grocery Bill ------ Item       Price      Qty        Total Rice
50.00      2          100.00 Milk       30.00      3          90.00 Sugar
45.00      1          45.00  Subtotal: 235.00 Tax (5%): 11.75 Grand Total:
246.75 ---------------------------
```

Gantt Chart

Task	Day 1	Day 2	Day 3-4	Day 5	Day 6
Planning	■■■■■				
Designing Classes		■■■■■			
Coding			■■■■■		
Testing				■■■■■	
Documentation				■■■■■	

Task	Duration
Planning	1 Day
Designing Classes	1 Day
Coding	2 Days
Testing	1 Day
Documentation	1 Day

Conclusion

The Grocery Billing Receipt Generator successfully demonstrates the practical implementation of Java programming and Object-Oriented Programming (OOP) concepts in solving a real-world retail problem. The system provides a simple, efficient, and automated approach to generating grocery bills, helping users add items, compute totals, apply taxes, and receive a clean formatted receipt with ease.

By incorporating features such as multiple item entry, automatic subtotal calculation, 5% tax computation, grand total display, and input validation, the application enhances billing accuracy and reduces manual effort. It also promotes better retail management by generating professional receipts that can be presented to customers.

The use of OOP principles ensures that the application is modular, maintainable, and scalable. The Item class encapsulates item attributes and behavior, while the Bill class demonstrates aggregation by managing a collection of Item objects. The use of ArrayList enables dynamic storage, and StringBuilder ensures efficient receipt text construction.

Although the current system is console-based with temporary in-memory data, it provides a strong foundation for future enhancements such as graphical user interfaces, database integration for persistent bill storage, discount management modules, and printer integration for physical receipts.

In conclusion, this project not only fulfills its objective of efficient grocery billing automation but also highlights the importance of structured programming and clean OOP design in developing practical and user-oriented applications. It serves as a valuable learning experience and a stepping stone toward building more advanced retail management and point-of-sale systems.

References

Books

[1] H. Schildt, *Java: The Complete Reference,* 11th ed. New York: McGraw-Hill, 2018.

[2] K. Sierra and B. Bates, *Head First Java,* 2nd ed. Sebastopol: O'Reilly Media, 2005.

[3] P. Deitel and H. Deitel, *Java: How to Program,* 10th ed. Pearson, 2015.

[4] E. Balagurusamy, *Programming with Java,* 6th ed. New Delhi: McGraw-Hill, 2019.

Websites

[5] Oracle, "Java Documentation," Oracle. [Online]. Available: https://docs.oracle.com

[6] GeeksforGeeks, "Java Programming Language," GeeksforGeeks. [Online]. Available: https://www.geeksforgeeks.org

[7] TutorialsPoint, "Java Tutorial," TutorialsPoint. [Online]. Available: https://www.tutorialspoint.com

[8] W3Schools, "Java Tutorial," W3Schools. [Online]. Available: https://www.w3schools.com/java/

Journal Articles

[9] A. Sharma and R. Gupta, "Machine Learning Techniques," *International Journal of AI,* vol. 5, no. 2, pp. 45-50, 2020.

[10] R. Kumar, "Data Analysis in Modern Applications," *Journal of Computer Science,* vol. 8, no. 3, pp. 112-118, 2019.

Conference Papers

[11] S. Kumar, "AI in Education," in *Proc. IEEE Conf. on Artificial Intelligence,* 2021, pp. 120-125.

[12] M. Verma, "Software Development Practices," in *Proc. Int. Conf. on Computing Systems,* 2020, pp. 78-84.